INVENTRA — SISTEMA DE INVENTARIO PARA RESTAURANTES

Informe Técnico del Hito #2

Proyecto Semestral | Equipo 06 | Integrantes: Wai Chung, Carlos Coba, Marlon Medina, Edison Centella

1. Flujo de una petición en el sistema INVENTRA

El ciclo de vida de una petición HTTP en la arquitectura de INVENTRA sigue un modelo monolítico modular en PHP. El navegador cliente inicia una solicitud (solicitud GET o POST), la cual es recibida y procesada por el servidor web Apache. El flujo detallado comprende las siguientes etapas:

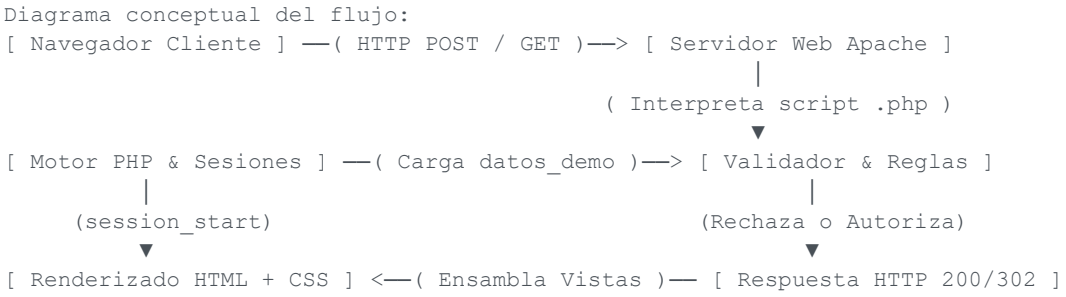
1. Solicitud cliente (Navegador): El usuario interactúa con la interfaz (ej. envía credenciales en inicio-sesion.php o registra una salida en registro-salida.php) emitiendo una petición HTTP hacia el servidor.

2. Servidor Web (Apache): Recibe la petición, evalúa la existencia del recurso .php solicitado y delega su procesamiento al intérprete de PHP mediante Mod_PHP / FastCGI.

3. Inicialización de Sesión (session_start): Antes de entregar contenido, el script valida las cookies de sesión (PHPSESSID) para autenticar al usuario e identificar su rol registrado ($_SESSION['rol_usuario']).

4. Procesamiento y Reglas de Negocio: El procesador (ej. procesar_salida.php) invoca a Validador.php y a funciones.php. Evalúa reglas complejas como verificar si el producto existe en datos_demo.php, comparar stock vs. mínimo y ejecutar determinarEstadoVencimiento().

5. Generación de Respuesta HTML: Si la validación es exitosa, se actualiza el estado simulado y se incluye el encabezado, menú dinámico (filtrado por rol) y pie de página, enviando un código de estado 200 OK con el HTML estructurado de vuelta al navegador.



2. Estructura de la aplicación web y responsabilidades

La estructura del proyecto fue organizada modularmente para separar la lógica de presentación, la lógica de negocio y la configuración de datos de prueba (simulando persistencia sin base de datos para este hito):

Directorio / Archivo	Propósito y Responsabilidad
config/datos_demo.php	Almacena los arreglos semilla $usuarios_demo y $productos_demo (contiene ítems del Hito #2 como PR-

	0407, PR-0408 y PR-0409).
includes/encabezado.php	Renderiza la cabecera HTML5, carga de estilos CSS y la barra superior con nombre de usuario activo y botón de salida.
includes/menu.php	Genera el menú de navegación lateral adaptativo, evaluando $_SESSION['rol_usuario'] para mostrar u ocultar accesos restringidos.
includes/pie.php	Contiene el cierre de etiquetas HTML, elementos de pie de página y metadatos de derechos.
includes/funciones.php	Librería de helper functions. Incluye limpiar() para sanitizar entradas y determinarEstadoVencimiento() para cálculo de fechas.
clases/Validador.php	Clase orientada a objetos encubridora de la lógica de validación de campos requeridos, formatos de correo, tipos numéricos y rangos.
procesar_inicio_sesion.php	Procesador de autenticación. Verifica credenciales contra datos_demo.php, inicia la sesión en $_SESSION y redirige según el rol.
procesar_salida.php	Procesador de salidas. Aplica la regla del Hito #2: rechaza movimientos si la cantidad es inválida o si el producto está 'Vencido'.
cerrar-sesion.php	Script de cierre seguro. Ejecuta session_unset() y session_destroy(), redirigiendo al usuario a inicio-sesion.php.

3. Tabla de validaciones del sistema

A continuación se detallan las reglas de validación implementadas en los formularios de la aplicación, indicando los campos evaluados, la regla aplicada y el mensaje retornado al usuario:

Formulario	Campo	Regla Aplicada	Mensaje de Error / Comportamiento
Inicio de Sesión	**usuario / contrasena**	Requerido (empty)	'El usuario o correo es obligatorio.' / 'La contraseña es obligatoria.'

Inicio de Sesión	**usuario**	Credenciales válidas	'Credenciales inválidas. Intente nuevamente.'
Producto	**nombre**	Requerido, minlength=2	'El nombre del producto es obligatorio (mínimo 2 caracteres).'
Producto	**codigo**	Formato Regex PR-[0-9]{3,}	'El código debe cumplir con el formato PR- seguido de al menos 3 dígitos.'
Producto	**precio**	Numérico, min=0.01	'El precio unitario debe ser un valor positivo mayor a 0.'
Producto	**stock_minimo**	Numérico, min=0	'El stock mínimo debe ser un número mayor o igual a 0.'
Registro Salida	**cantidad**	Numérico, min=0.01	'Value must be greater than or equal to 0.01.'
Registro Salida	**codigo_producto**	Regla Hito #2: Vencimiento	'Movimiento rechazado: El producto 'Queso fresco (kg)' se encuentra vencido.'

4. Decisiones de seguridad tomadas

Durante la transición de maquetas HTML a scripts PHP dinámicos se tomaron cinco decisiones de seguridad fundamentales:

A. Autenticación y Control de Accesos basado en Roles (RBAC): Se eliminó la definición estática de roles en las vistas. Ahora, cada página verifica la presencia de $_SESSION['rol_usuario']. Si un usuario sin autenticar intenta acceder a una vista protegida, es redirigido de inmediato a inicio-sesion.php. Si intenta acceder a un recurso no autorizado para su rol, se detiene la ejecución.

B. Preducción de Inyecciones XSS (Cross-Site Scripting): Todos los datos de salida renderizados dinámicamente en el HTML se procesan mediante la función htmlspecialchars($cadena, ENT_QUOTES, 'UTF-8'). Esto convierte caracteres especiales como <, >, &, ' y " en sus entidades HTML correspondientes, evitando la ejecución involuntaria de scripts maliciosos (verificado en pruebas S-04 y S-05).

C. Restricción estricta de Verbos HTTP (POST vs GET): Los archivos de procesamiento (procesar_inicio_sesion.php, procesar_producto.php, procesar_salida.php) verifican explícitamente $_SERVER['REQUEST_METHOD'] == 'POST'. Cualquier intento de invocar directamente el procesador vía URL (GET) es rechazado (prueba S-06).

D. Sanitización centralizada de datos de entrada: Se implementó la función helper limpiar() que aplica trim() y strip_tags() sobre las entradas POST antes de pasarlas a la capa de validación o lógica de negocio.

E. Cierre e Invalidación de Sesión de lado del Servidor: El script cerrar-sesion.php ejecuta session_unset() y session_destroy(), borrando la memoria en servidor y expirando la cookie PHPSESSID en el cliente.

5. Análisis para la publicación del sistema (Hosting, Dominio y BD)

Para desplegar el sistema INVENTRA en un entorno de producción accesible por pequeñas empresas de comida, se analizaron dos opciones de infraestructura web, considerando costos, escalabilidad y mantenimiento:

Criterio / Opciones	Opción 1: Shared Hosting (cPanel / LAMP)	Opción 2: Cloud VPS (Linux Ubuntu Server)
Proveedor Evaluado	Hostinger / Namecheap / Bluehost	DigitalOcean / Linode / AWS Lightsail
Costo Aproximado	$3.00 - $6.00 / mes	$6.00 - $12.00 / mes
Base de Datos	MySQL / MariaDB gestionado vía phpMyAdmin	MySQL / PostgreSQL instalado y optimizado manualmente
Dominio (.com / .pa)	$10.00/año (.com) / $35.00/año (.pa)	$10.00/año (.com) / $35.00/año (.pa)
Ventajas principales	Despliegue rápido por FTP/cPanel, SSL gratuito (Let's Encrypt), mantenimiento de servidor delegado.	Control total sobre la configuración de PHP, aislamiento de recursos, facilidades de integración continua (CI/CD) vía Git.
Desventajas	Recursos compartidos (memoria/CPU limitada), menor flexibilidad en versiones de paquetes.	Requiere conocimientos de administración de servidores Linux, seguridad de puertos y respaldos manuales.

Conclusión de Infraestructura: Recomendación de publicación:

Dado que INVENTRA está diseñado para pequeños restaurantes que buscan simplicidad y bajo costo operativo, la Opción 1 (Shared Hosting cPanel) es la más viable para la etapa inicial por su bajo costo (~$50/año total incluyendo dominio) y facilidad de administración.

6. Reflexión individual de cada integrante

Wai Chung (Líder de Proyecto y Repositorio): Durante este hito el principal desafío fue pasar de maquetas estáticas HTML a dinámicas por medio de PHP, integrando las funcionalidades necesarias con los scripts de procesamiento. En múltiples ocasiones enfrentamos dificultades al momento de validar las funcionalidades necesarias para asegurarnos que la lógica de negocio se respetara, mientras procurábamos mantener los componentes de seguridad indispensables. Durante la realización de los trabajos correspondientes pudimos darle forma al repositorio y corregir los inconvenientes que tuvimos durante la entrega del Hito1, logrando un resultado mucho más ordenado y consistente.

Carlos Coba (Diseño y Experiencia de Usuario): En este hito me enfrente al desafío de integrar las alertas dinámicas de vencimiento ('Por vencer' y 'Vencido') sin romper la estructura de diseño y las

clases CSS previamente maquetadas. En la primera prueba visual, la vista detalle-producto.php perdió su maquetación de tarjetas porque las variables inyectadas no utilizaban la estructura <dl class='eq06-detalle-grid'> original. Al inspeccionar el marcado HTML fuente y reestructurar los bloques PHP dentro de los contenedores <dt> y <dd>, logré que los badges de alerta se desplegaran respetando perfectamente la línea gráfica de la interfaz.

Marlon Medina (Desarrollo HTML5 y PHP): Mi trabajo se centró en la construcción del procesador de salidas y la implementación de las reglas del Hito #2. Un problema concreto que enfrenté fue que el script de salidas no estaba bloqueando el producto Queso fresco (PR-0409) a pesar de estar vencido. Al auditar la lógica, comprobé que el formulario no evaluaba el resultado de la función determinarEstadoVencimiento() antes de procesar el envío. Corregí procesar_salida.php inyectando una validación estricta al validador cuando el estado retorna 'Vencido', garantizando que el sistema rechace el movimiento con un error legible.

Edison Centella (Calidad, Seguridad y Documentación): Mi responsabilidad principal fue diseñar y ejecutar la batería de 18 casos de prueba manuales. Durante las pruebas de seguridad iniciales, detecté que era posible enviar scripts en los campos de entrada y que al intentar abrir procesar_producto.php directamente vía GET, el servidor arrojaba advertencias de PHP desordenadas. Para solucionar esto, comprobé el comportamiento asegurando que htmlspecialchars() sanitizara las salidas de texto y que los scripts procesadores redirigieran inmediatamente a inicio-sesion.php si la petición no era de tipo POST.